

Gymnázium, Praha 6, Arabská 14
předmět Programování, vyučující Mgr. Jan Lána

Plošinová hra „Útěk z ostrova“

ročníkový projekt



Poděkování

Rádi bychom poděkovali panu profesoru Mgr. Janu Lánovi, který byl vedoucí naše ročníkové práce. Děkujeme mu za pomoc a zpětnou vazbu během vypracování tématu, rady a nápady k obsahu práce.

Anotace

Ve své práci se snažíme o vytvoření vypečené plošinové hry jménem „Útěk z ostrova“. Práce zahrnuje využití dovedností v oboru programování, zejména v jazyku C++, v počítačové pixelové grafice, propojení obou částí a uvedení hry do funkce. Dokumentace obsahuje popis procesu vytvoření hry a její jednotlivých složek, vysvětlení jak hra funguje, ukázky samotné hry a kódu.

Abstract

In this project, we aim to create a polished platformer game titled "Escape from the Island." The project involves applying skills in programming, particularly in C++, in computer pixel graphics, integrating both components, and bringing the game to life. The documentation includes a description of the game development process and its individual components, an explanation of how the game works, and examples of the game itself and the code.

Anmerkung

In unserer Arbeit versuchen wir, ein ausgefeiltes Plattformspiel mit dem Titel „Flucht von der Insel“ zu entwickeln. Die Arbeit umfasst die Anwendung von Programmierkenntnissen, insbesondere in der Sprache C++, sowie Kenntnisse in der computergestützten Pixelgrafik, die Verknüpfung beider Bereiche und die Inbetriebnahme des Spiels. Die Dokumentation enthält eine Beschreibung des Entstehungsprozesses des Spiels und seiner einzelnen Komponenten, eine Erläuterung der Funktionsweise des Spiels sowie Beispiele für das Spiel selbst und den Code.

Résumé

Dans ce projet, nous nous efforçons de créer un jeu de plateforme bien ficelé intitulé « Évasion de l'île ». Ce travail fait appel à des compétences en programmation, notamment en langage C++, en infographie pixelisée, à l'intégration de ces deux aspects et à la mise en service du jeu. La documentation comprend une description du processus de création du jeu et de ses différents composants, une explication de son fonctionnement, ainsi que des exemples du jeu et du code.

Prohlášení

Prohlašujeme, že jsme jediní autoři tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů udělujeme bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne _____

Vlastnoruční podpis autorů

Obsah

Úvod.....	6
1. Gameplay.....	7
2. Návrh a struktura hry.....	8
2.1 Hlavní komponenty systému.....	8
2.2 Tok programu.....	9
3. Programování hry.....	10
3.1 Použité technologie.....	10
3.2 Herní smyčka.....	11
3.3 Herní logika.....	11
3.4 Kolize a interakce.....	11
4. Grafika.....	12
4.1 Vizuální motiv.....	12
4.2 Proces vytváření grafiky.....	13
4.3 Propojení grafiky s kódem.....	13
5. Uvedení hry do funkce.....	14
6. Jak nainstalovat?.....	15
Závěr.....	16
Zdroje.....	17

Úvod

Hra „Útěk z ostrova“ je singleplayerová hra typu plošinovka. Hra je vytvořena ve 2D prostředí a využívá pixelovou grafiku.

Hra obsahuje jednu úroveň, jejíž obtížnost se postupně zvyšuje během postupování hráče. Cílem hráče je dostat se ze začátku úrovně do cíle a překonat všechny překážky, které se na cestě objeví. Úroveň se bude postupně ztěžovat – budou se objevovat nové překážky a herní mechaniky, jakmile hráč postupuje dál. Hráč si tak bude muset osvojit logiku hry, zapamatovat si rozmístění překážek a najít způsob, jak úroveň úspěšně projít.

Ročníková práce je zároveň základem pro jednoduchý engine, je možné projekt použít pro vytvoření dalších her. Další hry je možné vytvořit prostřednictvím změny obrázků lokací a textur.

1. Gameplay

Hra je 2D plošinovka kde se hráč se snaží uniknout z podzemního prostředí opuštěného ostrova postupným překonáváním jednotlivých levelů.

Základním principem hry je pohyb postavy v prostředí s překážkami. Hráč musí správně načasovat pohyb, skoky a reakce na herní prvky, aby se dostal bezpečně na druhý konec úrovně. Mezi hlavní herní mechaniky patří:

- **pohyb do stran (vlevo a vpravo)**
- **skoky**
- **vyhýbání se nástrahám**

Úrovně jsou navrženy tak, aby představovaly výzvu. Na začátku jsou úrovně jednodušší a slouží k seznámení s ovládáním a základními principy hry. Postupně se obtížnost zvyšuje a hráč je nucen reagovat rychleji a přesněji.

Cílem hráče je projít jednotlivé levely a dostat se na jejich konec bez ztráty života nebo pádu do nebezpečných zón. Úspěšné dokončení levelu umožňuje postup do další části hry. Cílem celé hry je dostat se ven z podzemí, kde hra končí.

2. Návrh a struktura hry

Při návrhu hry byl kladen důraz především na modularitu, přehlednost a možnost budoucího rozšiřování projektu. Cílem nebylo vytvořit pouze jednoduchou 2D hru, ale navrhnout základní architekturu, která umožní oddělit jednotlivé části programu a zajistit jejich vzájemnou spolupráci. Projekt je proto rozdělen do několika samostatných systémů odpovědných za správu grafiky, herních objektů, vstupů, konfigurace a herní logiky.

Architektura programu je postavena na vlastních datových strukturách označovaných jako „*clustery*“. Tyto struktury slouží jako kontejnery pro různé typy dat a umožňují organizovat jednotlivé části programu jednotným způsobem. Díky tomuto přístupu je možné jednoduše spravovat entity, textury i renderovací prostředky bez nutnosti vytváření velkého množství oddělených proměnných nebo globálních struktur.

Velká část návrhu byla přizpůsobena knihovně *SDL3*, která zajišťuje komunikaci s hardwarem, práci s okny, vykreslování i zpracování vstupů. Návrh systému zároveň počítá s možností budoucí implementace dalších herních mechanik, například složitější fyziky, nových typů entit nebo propracovanější správy levelů. Jednotlivé části programu jsou navrženy tak, aby bylo možné jejich funkcionalitu rozšiřovat bez zásadních zásahů do zbytku projektu.

Součástí návrhu je také oddělení logické a grafické části programu. Herní logika pracuje pouze s daty a stavem objektů, zatímco vykreslovací systém zajišťuje jejich vizuální reprezentaci. Tento přístup zvyšuje přehlednost zdrojového kódu a usnadňuje následnou údržbu programu.

2.1 Hlavní komponenty systému

Architektura hry je navržena modulárně a je rozdělena do několika samostatných systémů, které spolu vzájemně komunikují. Hlavním cílem tohoto návrhu bylo oddělit jednotlivé části programu tak, aby bylo možné projekt snadno rozšiřovat a udržovat.

Základním prvkem je systém správy okna a rendererů reprezentovaný strukturou *WRCluster*. Tato struktura uchovává hlavní *SDL* okno a dynamické pole rendererů, které jsou následně využívány při vykreslování herní scény. Renderer je používán prakticky ve všech částech programu souvisejících s grafikou.

Další důležitou částí je systém entit realizovaný pomocí struktury *ECluster*. Jedná se o dynamický kontejner obsahující všechny entity přítomné ve hře. Každá entita představuje konkrétní objekt herního světa, například hráče, pozadí, překážky nebo menu. Přístup k entitám je realizován pomocí přetíženého operátoru indexace.

Samotná struktura *entity* obsahuje informace o pozici objektu, jeho velikosti, směru pohybu, rychlosti, životě, poškození a dalších stavech. Součástí *entity* je také struktura *TCluster*, která zajišťuje správu textur a animací. Díky tomu lze oddělit logickou reprezentaci objektu od jeho grafického vykreslení.

Zpracování vstupu od uživatele zajišťuje struktura *user_events*. Ta převádí *SDL* události na jednodušší logické proměnné, například stav jednotlivých kláves nebo pozici myši. Tento přístup výrazně zjednodušuje následnou implementaci herní logiky.

Projekt dále obsahuje samostatný systém konfigurace (*ConfigFile*), který umožňuje načítání parametrů z externího souboru *config.ini*. Program tak není závislý pouze na hodnotách zapsaných přímo ve zdrojovém kódu. Součástí projektu je také logovací systém (*RuntimeLog*) a systém pro zpracování chyb (*ErrorHandle*).

Celý program je rozdělen do několika navzájem propojených částí. Jednotlivé systémy si mezi sebou předávají data prostřednictvím sdílených struktur a společně vytvářejí výsledné chování hry.

Hlavní herní smyčka získává vstupy od uživatele pomocí struktury *user_events*. Tyto vstupy jsou následně předány herní logice implementované v prostoru jmen *GameConditions*. Herní logika upravuje stav jednotlivých entit, například jejich pohyb, animace nebo interakce s prostředím.

Po aktualizaci logiky dochází ke změně pozic objektů a případnému přepnutí aktivních textur. Výsledný stav je následně předán renderovacím funkcím, které jednotlivé entity vykreslí na obrazovku. Tento proces se neustále opakuje po dobu běhu hry.

Hra využívá také pomocné hitboxy reprezentované strukturou *HCluster*, které slouží zejména pro kolizní systém. Hitboxy definují části mapy, se kterými mohou entity interagovat, například podlahu, překážky nebo přechody mezi úrovněmi.

2.2 Tok programu

Program začíná ve funkci *WinMain*, která představuje vstupní bod celé aplikace. Nejprve dojde k vytvoření logovacího souboru a inicializaci knihovny *SDL*. Pokud inicializace selže, program je ukončen pomocí systému pro zpracování chyb.

Po úspěšné inicializaci jsou vytvořeny hlavní datové struktury programu, tedy okno, renderery, entity, kontejner pro uživatelské vstupy a struktura reprezentující aktuální stav hry. Následně program načte konfiguraci, vytvoří entity a připraví jejich textury.

Po dokončení inicializace je spuštěna hlavní herní smyčka. Ta opakovaně zpracovává vstupy od uživatele, aplikuje herní logiku, aktualizuje pozice objektů a vykresluje scénu. Smyčka běží až do okamžiku, kdy uživatel požádá o ukončení programu nebo je aktivováno ukončení z herního menu.

Po ukončení hlavní smyčky program korektně zničí renderery, okno a ukončí *SDL* subsystémy. Nakonec jsou do logu zapsány informace o úspěšném ukončení programu.

3. Programování hry

Programování hry probíhalo v jazyce C++ s využitím knihoven *SDL3* a *SDL3_image*. Při vývoji byl kladen důraz nejen na samotnou funkčnost hry, ale také na návrh vlastní architektury a pochopení principů, které se využívají při vývoji větších herních projektů. Výsledný program tak není pouze jednoduchou hrou, ale zároveň představuje základ vlastního herního frameworku.

Velká část implementace byla zaměřena na správu herních objektů, práci s texturami a návrh herní smyčky. Program využívá vlastní struktury pro organizaci dat a propojuje několik různých subsystémů, které spolu během běhu aplikace komunikují. Důležitou součástí implementace je také práce s externí konfigurací a dynamické načítání grafických assetů.

Při vývoji byly využity moderní prvky jazyka C++, například standardní knihovny pro práci se soubory, kontejnery a správu datových struktur. Program současně využívá i některé pokročilejší koncepty, například oddělení logiky od renderingu nebo základní paralelní zpracování pomocí vláken. Tyto prvky umožňují lepší organizaci programu a připravují projekt na případné další rozšiřování.

Důležitou součástí implementace bylo také řešení stability programu a ošetření chybových stavů. Projekt obsahuje vlastní logovací systém, který zaznamenává důležité události během běhu aplikace, a systém pro základní ošetření chyb při inicializaci *SDL* nebo načítání souborů. Díky tomu je možné jednodušeji identifikovat problémy vzniklé během vývoje nebo testování.

Celý projekt byl navrhován s důrazem na pochopení principů herního vývoje a získání zkušeností s tvorbou komplexnější aplikace. Vývoj hry tak sloužil nejen k vytvoření funkčního programu, ale také k procvičení návrhu architektury, organizace zdrojového kódu a práce s multimediálními knihovnami.

3.1 Použité technologie

Projekt byl implementován v programovacím jazyce C++ s využitím knihovny *SDL3* a její rozšiřující knihovny *SDL3_image*. Jazyk C++ byl zvolen především kvůli vysokému výkonu, možnosti nízkoúrovňové práce s pamětí a podpoře objektově orientovaného programování.

Knihovna *SDL3* zajišťuje práci s grafickým oknem, vstupy z klávesnice a myši, vykreslováním textur i komunikací s hardwarem. Knihovna *SDL3_image* je využívána pro načítání *PNG* souborů, které obsahují grafické assety hry.

Projekt využívá také standardní knihovny jazyka C++, například *vector*, *array*, *set*, *filesystem*, *fstream* a *sstream*. Tyto knihovny slouží ke správě datových struktur, práci se soubory a zpracování textových konfigurací.

Součástí projektu je rovněž vlastní systém konfigurace, logování a zpracování chyb. Konfigurační systém umožňuje měnit parametry programu bez nutnosti rekompilace, zatímco logovací systém zaznamenává důležité události během běhu programu.

3.2 Herní smyčka

Hlavní herní smyčka představuje základní mechanismus celé hry a je implementována ve funkci *MainLoop*. Tato smyčka běží nepřetržitě po dobu běhu aplikace a řídí všechny důležité části programu.

V každé iteraci smyčky dochází nejprve ke zpracování vstupů od uživatele. Program následně aplikuje herní logiku pomocí funkcí z prostoru jmen *GameConditions*. Tyto funkce řeší například pohyb hráče, skákání, kolize, interakci s překážkami nebo přechod mezi levely.

Po zpracování herní logiky jsou aktualizovány pozice objektů a případně změněny jejich textury. Nakonec dojde k vykreslení všech renderovatelných entit na obrazovku pomocí *SDL* rendereru.

3.3 Herní logika

Herní logika je implementována v samostatném prostoru jmen *GameConditions*. Tento přístup odděluje logickou část programu od renderingu a správy datových struktur, což zvyšuje přehlednost projektu.

Systém obsahuje několik funkcí zajišťujících různé části chování hry. Patří mezi ně například pohyb hráče (*PlayerMovement*), skákání (*PlayerJump*), kolizní systém (*PlayerCollisions*) nebo interakce s pohyblivými překážkami. Implementována je také logika menu a změny herních lokací.

Pohyb objektů je realizován pomocí směrového vektoru uloženého v každé entitě. Program následně upravuje pozici hitboxu podle aktuálních hodnot vektoru. Tento systém umožňuje jednoduché a přehledné řízení pohybu.

3.4 Kolize a interakce

Kolizní systém je založen na práci s hitboxy reprezentovanými strukturou *SDL_FRect*. Každá entita obsahuje vlastní *hitbox* definující její pozici a velikost v herním prostoru.

Detekce kolizí probíhá porovnáváním souřadnic jednotlivých hitboxů. Program dokáže určit nejen obecný překryv objektů, ale také směr kolize, například zda došlo ke kontaktu zleva, zprava nebo shora.

Kolize jsou využívány při pohybu hráče, interakci s překážkami i při změnách levelů. Součástí systému je také základní mechanismus poškození a práce se životy entit.

4. Grafika

Tato kapitola se zabývá vysvětlením a nastíněním grafické části celé hry. Grafika hry je nezbytnou součástí nejen našeho projektu, ale i každé počítačové hry. Má za cíl zapůsobit na hráče, zpříjemnit herní zkušenost a dodat hře estetickou a uměleckou složku.

Grafika se dělí na pixelovou (rastrovou) a vektorovou. V této ročníkové práci používáme pixelovou grafiku, to znamená, že obraz je tvořen za pomoci jednotlivých pixelů (čtverečků), každý z nich jde zvlášť obarvit. Daný typ grafiky jsme vybrali, protože se používá pro tvorbu her, složitějších obrázků (lokace, postavy apod.). Grafika celé hry byla vytvořena v aplikaci Pixel studio a celá je zpracována ve stylu pixel art.

4.1 Vizuální motiv

Práce zahrnuje návrh a tvorbu herních levelů, postav a překážek. Cílem bylo vytvořit vizuální styl odpovídající prostředí opuštěného podzemí ostrova a také scénérii samotného ostrova.

Celá hra je navržena v jednodušším 2D stylu. Tento přístup byl zvolen především kvůli přehlednosti během hraní a také kvůli menší technické náročnosti a zároveň je vizuálně atraktivní. Barevná paleta se skládá primárně z barev, které mají přírodní objekty, tedy primárně zelená, šedá a hnědá.

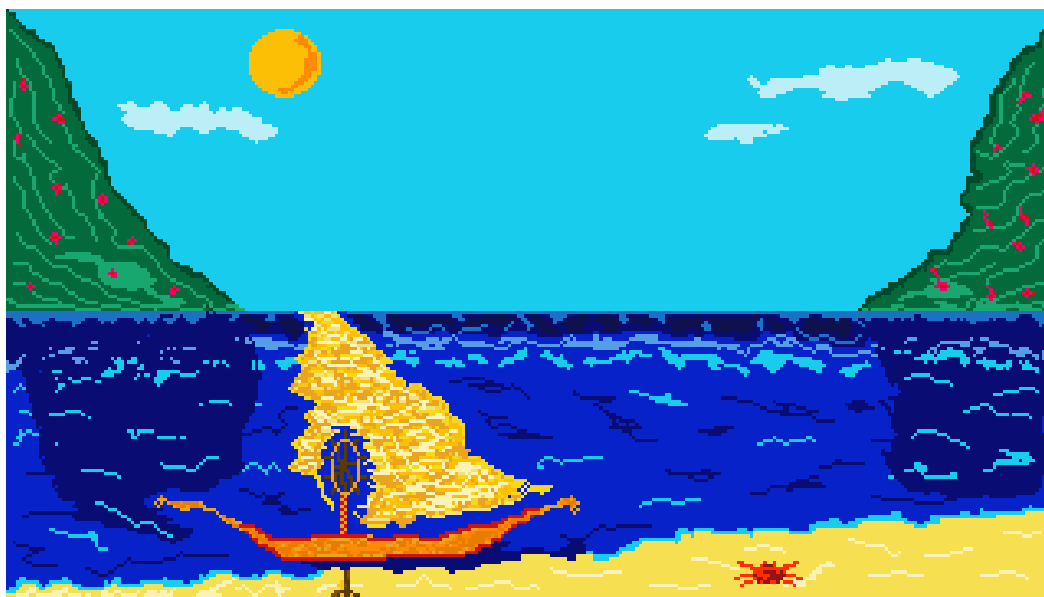
Hra využívá objekty typické pro prostředí opuštěného ostrova, jako jsou kameny, opuštěné doly, tunely nebo stalagmity. Nástrahy a překážky jsou tvořeny právě těmito přírodními prvky, čímž je zachována jednotná atmosféra celého herního prostředí.



[Obrázek 1] Ukázka pozadí levelu

Finální lokace hry, což je zároveň cíl hráče, byla vymyšlená a inspirovaná fantasy filmy, skutečnými rostlinami z botanické zahrady v Krakově a trochu kulturou pirátů. Před hráčem se objevuje krajina s mořem, pláží, lodí, lítajícím ostrovem a tropickými horami. Dominantní barvy jsou modrá a její odstíny, olivová, hořčicová a oranžová. Krajina symbolizuje svobodu, začátek něčeho nového, budoucnost.

Nejdříve jsem lokaci vykreslila ručně a následně to zrealizovala prostřednictvím aplikace Pixel studio.



[Obrázek 2] Lokace ostrova

4.2 Proces vytváření grafiky

Proces tvorby grafiky probíhal v několika fázích, od prvotního návrhu až po finální implementaci do hry. Po stanovení jasného vizuálního motivu a stylu hry byly vytvořené jednoduché návrhy postav a levelů. Také byly vytvořeny náčrtky ostrova pro lepší představu místa a následně jednodušší realizaci v aplikaci Pixel Studio. Náčrtky jsem vytvářela nejdřív rukou a potom i v programu. Po fázi náčrtku jsem přidávala více konkrétních detailů na obraz ostrova. Pro lepší a přesnější vykreslení elementů lokace, jsem používala geometrické útvary, jakožto obdélník, čára, kruh.

Při práci s pixelart grafikou bylo využíváno škálování (upscaling), kdy byly menší obrázky (např. 32×32 px) zvětšovány na vyšší rozlišení (např. 128×128 px). Byla použita metoda „*nearest-neighbor*“, která zachovává ostré hrany a typický vzhled pixel artu bez rozmazání.

Po vytvoření základních verzí grafiky následovalo jejich testování přímo ve hře. Na základě zpětné vazby ze hry byla upravována například:

- velikost objektů
- barevnost
- čitelnost postav a překážek
- rozlišení

4.3 Propojení grafiky s kódem

Grafická část hry je přímo propojena s programovým kódem pomocí systému dynamického načítání textur a jejich přiřazování jednotlivým entitám. Veškeré grafické assety jsou uloženy v externích *PNG* souborech, které program načítá při spuštění hry. Tento přístup umožňuje jednodušší správu grafiky a zároveň usnadňuje výměnu nebo úpravu jednotlivých assetů bez nutnosti zásahu do zdrojového kódu.

Po načtení obrázku je vytvořen *SDL_surface*, který je následně převeden na *SDL* texturu použitelnou při vykreslování. Program podporuje také škálování obrázků, přičemž je použita metoda „*nearest-neighbor*“, která zachovává ostré hrany typické pro pixelartovou grafiku. Výsledné textury jsou ukládány do struktury *TCluster*, která umožňuje organizovat textury do skupin a efektivně mezi nimi přepínat.

Každá entita obsahuje vlastní instanci struktury *TCluster*, díky čemuž lze jednotlivým objektům přiřadit odlišné textury nebo animace. Tento systém je využíván například u hráče, kde dochází ke změně aktivní textury podle aktuálního směru pohybu nebo stavu entity. Program tak odděluje logiku objektu od jeho vizuální reprezentace, což zvyšuje přehlednost celého projektu. Renderování probíhá pomocí *SDL* rendereru, který vykresluje aktivní textury na základě aktuální pozice a velikosti hitboxu entity. Díky tomuto propojení je možné jednoduše aktualizovat vzhled objektů během běhu hry a vytvářet základní animace nebo vizuální efekty.

5. Uvedení hry do funkce

Spuštění programu probíhá prostřednictvím funkce *WinMain*, která slouží jako vstupní bod aplikace. Na začátku je vytvořen logovací soubor, do kterého jsou zapisovány důležité informace o běhu programu. Následně dojde k inicializaci knihovny *SDL*, bez které by nebylo možné pracovat s grafikou ani vstupy.

Po úspěšné inicializaci je vytvořeno hlavní okno aplikace a renderer, který zajišťuje vykreslování. Program poté načte konfigurační soubor, ze kterého získá potřebné informace o nastavení hry, například rozměry okna nebo umístění grafických souborů. Následuje inicializace všech entit a načtení jejich textur.

Jakmile jsou všechny komponenty připraveny, je spuštěna hlavní herní smyčka. Po ukončení této smyčky, například při zavření okna, program korektně ukončí všechna vlákna, zničí renderer i okno a uvolní všechny použité prostředky.

6. Jak nainstalovat?

Pro spuštění hry je nejprve nutné zajistit, aby byly všechny potřebné soubory umístěny ve správné složce. Jedná se zejména o spustitelný soubor programu, konfigurační soubor a složku obsahující grafické assety. Pokud je projekt distribuován jako archiv, je potřeba jej nejprve rozbalit.

Po přípravě všech souborů stačí spustit výsledný „*exe*“ soubor. Program si při spuštění automaticky načte potřebná data a inicializuje herní prostředí. Pro správný běh je vyžadován operační systém Windows a podpora knihoven *SDL3* a *SDL3_image*. Dále je nutná funkční grafická karta, která umožňuje vykreslování pomocí GPU.

Závěr

Cílem této práce bylo vytvořit funkční 2D hru v programovacím jazyce C++ s využitím knihovny *SDL3* a současně si osvojit principy návrhu komplexnějších softwarových projektů. Během vývoje se podařilo implementovat základní herní mechaniky, systém správy entit, práci s texturami, herní logiku i vlastní architekturu programu.

Projekt se během vývoje postupně rozšířil nad rámec jednoduché školní hry a začal obsahovat prvky připomínající základ vlastního herního enginu. Byly implementovány vlastní datové struktury pro správu objektů a textur, systém konfigurace, logování i oddělení logické a grafické části programu. Součástí projektu je také základ paralelního zpracování a připravená infrastruktura pro další rozšiřování hry.

Při práci na projektu byly získány zkušenosti nejen s programováním v jazyce C++, ale také s návrhem architektury aplikací, správou paměti, organizací zdrojového kódu a prací s multimediálními knihovnami. Vývoj hry zároveň ukázal některé problémy spojené s tvorbou větších projektů, například synchronizaci dat nebo správu sdílených prostředků.

Do budoucna by bylo možné projekt dále rozšiřovat například o propracovanější fyzikální systém, pokročilejší správu kolizí, více herních úrovní, zvuky, menu nebo editor map. Současná verze však již představuje funkční základ, na kterém lze dále stavět a který splnil stanovené cíle práce.

Zdroje

SDL3 dokumentace. [online] [3. 5. 2026]. Dostupné z < <https://wiki.libsdl.org/SDL3/FrontPage>

>

SDL_image dokumentace. [online] [3. 5. 2026]. Dostupné z <

https://wiki.libsdl.org/SDL3_image/FrontPage >

C++ reference. [online] [3. 5. 2026]. Dostupné z < <https://cppreference.com/> >

vlastní zdrojové kódy. [online] [3. 5. 2026]. Dostupné z < https://github.com/gyarab/2025-3e-utek_z_ostrova >

Seznam obrázků, grafů a tabulek

Obrázek 1 – Ukázka pozadí levelu. [str. 12]

Obrázek 2 – Lokace ostrova [str. 13]

Použité technologie

- Vývoj softwaru:
 - Microsoft Visual Studio 2022 – jazyk C++
- Pracovní software: